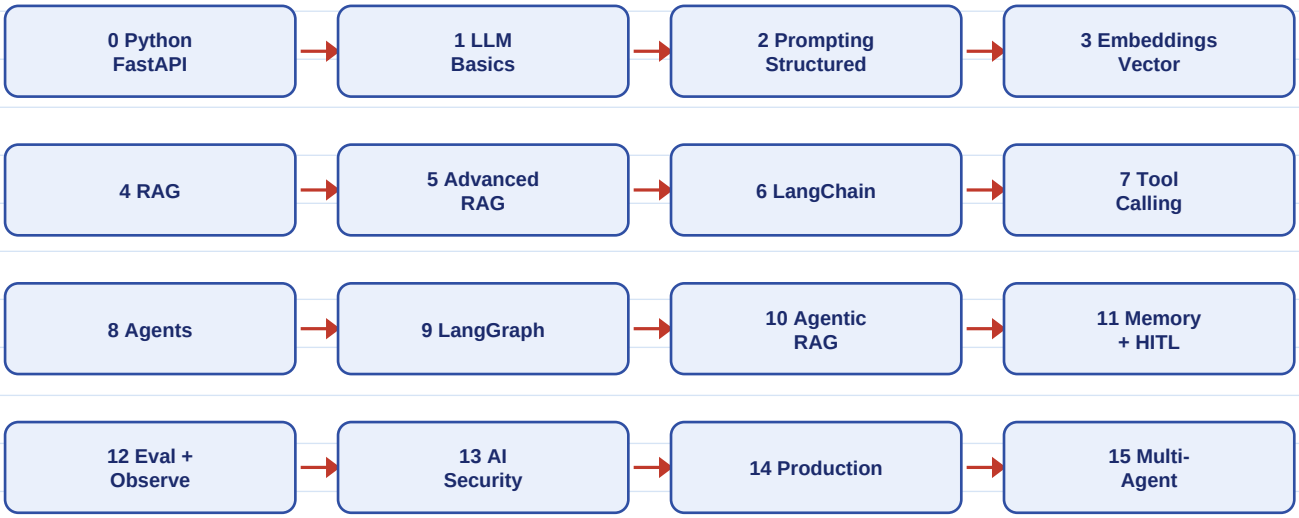
GenAI & Agentic AI

Progress Answer Sheet - MERN to Agentic AI

Name: _____ Start date: _____ Target date: _____

Q. Roadmap at a glance (complete in order; move on only when the 'Move on when' line is true)



Then Phase 16 - Major Project (one serious AI Project Intelligence Agent).

How to use this sheet

Tick the box after finishing each topic. Fill the start and completion date for every phase. Follow this cycle for every topic: **Watch -> Understand -> Implement tutorial -> Close tutorial -> Rebuild yourself -> Modify -> Build something new.** Progress means 'I can build this without the tutorial', not 'I watched the video'.

PHASE 0 - PYTHON FOR GENAI BACKEND

Time: 3-5 days

Started: _____ Completed: _____

- [ ] **1. Virtual environments & pip**
  A venv gives each project its own isolated libraries so versions never clash. pip installs the packages. Create one before every new project.
- [ ] **2. .env & environment variables**
  API keys live in a .env file loaded with python-dotenv, never inside code. This keeps secrets out of GitHub.
- [ ] **3. requests & JSON**
  Calling APIs over HTTP and reading/writing JSON. Every LLM API is just an HTTP endpoint that takes and returns JSON.
- [ ] **4. async / await**
  Lets the server handle other requests while waiting for a slow LLM call. Needed later for streaming and parallel calls.
- [ ] **5. Exception handling & type hints**
  try/except for failed or rate-limited API calls; type hints make code readable and are used by Pydantic and FastAPI.
- [ ] **6. Pydantic**
  Python classes that validate data (request bodies, LLM outputs). The base for structured output in Phase 2.
- [ ] **7. FastAPI & REST APIs**
  Python web framework for building endpoints with automatic validation and docs. This is the AI backend that React/Express will call.

Build: POST /chat endpoint that sends a prompt to an LLM and returns the reply.

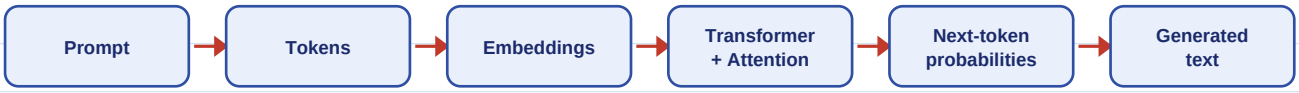
Move on when: I can build a FastAPI server, validate JSON with Pydantic, call an LLM API, use env variables and handle API errors.

My notes: _____

PHASE 1 - LLM FUNDAMENTALS

Time: 5-7 days

Started: _____ Completed: _____



- [ ] **1. What is an LLM**
  A model trained on huge text data to predict the next token. It generates text one token at a time.
- [ ] **2. Tokens & tokenization**
  Text is split into small pieces (tokens) before the model sees it. Cost, speed and limits are all measured in tokens.
- [ ] **3. Context window**
  Maximum tokens (prompt + history + answer) the model can handle at once. Anything beyond it is forgotten or must be trimmed.
- [ ] **4. Temperature & top-p**
  Settings controlling randomness. Low temperature gives stable, factual output; high gives creative, varied output.
- [ ] **5. Hallucination**
  The model confidently produces wrong or invented information. RAG and tools are used to reduce it.
- [ ] **6. System / user / assistant messages**
  Roles in a chat. System sets behaviour, user asks, assistant replies. Conversation history is just a list of these.
- [ ] **7. Transformer & attention (concept only)**
  Tokens become embeddings, attention lets each token look at the others, and the model outputs next-token probabilities. Know encoder vs decoder, positional info. Do not implement it.

- [ ] **8. Pretraining, fine-tuning, instruction tuning**
  Pretraining learns language, instruction tuning teaches following commands, fine-tuning adapts to a task or domain.
- [ ] **9. LLM API (start with Gemini)**
  Learn chat calls, system instructions, streaming, max tokens, model choice, errors and rate limits using one provider first.

Build: AI chat app with streaming, conversation history, system prompt and error handling.

Move on when: I can explain what happens from sending a prompt until the response arrives.

My notes: _____

PHASE 2 - PROMPT ENGINEERING + STRUCTURED OUTPUT

Time: 3-4 days

Started: _____ Completed: _____

- [ ] **1. Prompt structure**
  Role + Task + Context + Constraints + Examples + Output format. A clear structure gives far more reliable answers.
- [ ] **2. Zero-shot & few-shot prompting**
  Zero-shot gives only the instruction; few-shot adds examples so the model copies the pattern and format.
- [ ] **3. Role prompting & prompt templates**
  Assigning a role shapes tone and depth. Templates with placeholders make prompts reusable in code.
- [ ] **4. Delimiters & output constraints**
  Separate instructions from data using markers like ``` or ###. State length, format and what not to do.
- [ ] **5. Task decomposition & context injection**
  Break a big task into smaller steps and feed only the relevant context into the prompt.
- [ ] **6. Structured output (JSON, schema, Pydantic)**
  Force the LLM to return machine-readable JSON matching a schema, then validate it with Pydantic. Essential for real applications.

Build: AI Resume Analyzer: input resume + job description, output match_score, skills, missing_skills, strengths, improvements.

Move on when: I can reliably get machine-readable output from an LLM.

My notes: _____

PHASE 3 - EMBEDDINGS + VECTOR SEARCH

Time: 4-5 days

Started: _____ Completed: _____

- [ ] **1. Embeddings**
  A model converts text into a vector of numbers so that similar meanings sit close together in vector space.
- [ ] **2. Vectors & dimensions**
  Each embedding is a list of numbers (e.g. 768). Same embedding model must be used for documents and queries.
- [ ] **3. Cosine similarity**
  Measures the angle between two vectors; closer to 1 means more similar meaning. The core of similarity search.
- [ ] **4. Semantic vs keyword search**
  Keyword search matches exact words; semantic search matches meaning, so 'extend deadline' finds 'time extension rules'.

- [ ] **5. Vector database (PostgreSQL + pgvector)**
  Store embeddings in a vector column and query nearest neighbours with SQL. Pick ONE database and learn it well.

- [ ] **6. Metadata & filtering**
  Store extra fields (user_id, type, date) with each vector and filter on them during search.

Build: Semantic search engine over 10-20 documents.

Move on when: I can explain why semantic search is different from keyword search.

My notes: _____

PHASE 4 - RAG * — Time: 1-1.5 weeks

** Core skill*

Started: _____ Completed: _____

- [ ] **1. RAG pipeline overview**
  Retrieval-Augmented Generation: fetch relevant text from your own data and give it to the LLM so answers are grounded, current and less hallucinated.

- [ ] **2. Document parsing**
  Extract clean text from PDF, DOCX, TXT, HTML and Markdown. Bad parsing means bad answers later.

- [ ] **3. Chunking**
  Split documents into pieces: fixed-size, sentence-based or semantic. Overlap keeps context across boundaries. Understand why, not just chunk_size=500.

- [ ] **4. Embedding & storing chunks**
  Embed each chunk and store it in the vector DB with metadata like source and page.

- [ ] **5. Retrieval**
  Embed the query, run similarity search, take top-k chunks, apply a similarity threshold and metadata filter.

- [ ] **6. Context construction & answer generation**
  Put the retrieved chunks and the question into a prompt that tells the LLM to answer only from the context.

- [ ] **7. RAG failure cases**
  Bad chunks, irrelevant retrieval, missing information, too much context, duplicate chunks, and hallucination despite context.

Build: Document Intelligence System: React + FastAPI + pgvector + LLM. Upload PDFs and ask questions.

Move on when: I can explain every step of RAG without looking at notes.

My notes: _____

PHASE 5 - ADVANCED RAG — Time: 1 week

Started: _____ Completed: _____

- [ ] **1. Metadata filtering**
  Restrict search by project_id, user_id or document_type so only the right data is retrieved.

- [ ] **2. Query rewriting**
  Turn vague questions ('Why did they reject it?') into complete, searchable queries using the LLM.
- [ ] **3. Multi-query retrieval**
  Generate several versions of one question, search for each, and merge results for better recall.
- [ ] **4. Hybrid search**
  Combine keyword search (BM25) with vector search so both exact terms and meaning are matched.
- [ ] **5. Reranking**
  Retrieve about 20 chunks, then a reranker picks the best 5 before sending to the LLM.
- [ ] **6. Citation-aware RAG**
  Carry chunk -> source -> page -> document through the pipeline so answers show [Source: file, page].
- [ ] **7. Access-controlled RAG**
  User -> authorization -> retriever filter -> allowed documents only. User A must never retrieve User B's data.

Build: Upgrade the Phase 4 system with metadata, citations, filtering, better retrieval and permission boundaries.

Move on when: My RAG is more than 'chat with PDF'.

My notes: _____

PHASE 6 - LANGCHAIN

Time: 4-5 days

Started: _____ Completed: _____

- [ ] **1. Models & prompt templates**
  Unified interfaces to call different LLMs and manage reusable prompts.
- [ ] **2. Output parsers**
  Convert LLM text into JSON or Pydantic objects automatically.
- [ ] **3. Document loaders & text splitters**
  Ready-made tools to load files and chunk them.
- [ ] **4. Embeddings & retrievers**
  Wrappers for embedding models and vector stores that return relevant documents for a query.
- [ ] **5. Chains (LCEL)**
  Connect steps like Prompt -> LLM -> Parser or Retriever -> Prompt -> LLM into one pipeline.
- [ ] **6. Tools & callbacks**
  Define tools the LLM can call; callbacks let you log and trace each step.

Build: Rebuild your RAG pipeline with LangChain after doing it manually.

Move on when: I can build a basic RAG without LangChain and then simplify it with LangChain.

My notes: _____

PHASE 7 - TOOL CALLING *

Time: 4-5 days

** Bridge to agents*

Started: _____ Completed: _____

- [ ] **1. Tool schema**
  A description of each function (name, purpose, parameters) given to the LLM so it knows what it can call.

- [ ] **2. Tool arguments & validation**
  The LLM produces arguments in JSON; always validate them before running anything.
- [ ] **3. Tool execution & result**
  Your code runs the function and sends the result back to the LLM, which writes the final answer.
- [ ] **4. Multiple tools**
  Give the LLM several tools and let it choose: calculator, search_documents, get_project, query_database.
- [ ] **5. Tool errors**
  Handle failures, bad arguments and empty results gracefully and let the LLM recover.
- [ ] **6. Permission checks**
  Check user rights before executing sensitive tools like create_task or send_notification.

Build: LLM that chooses among 4-5 tools based on the user's request.

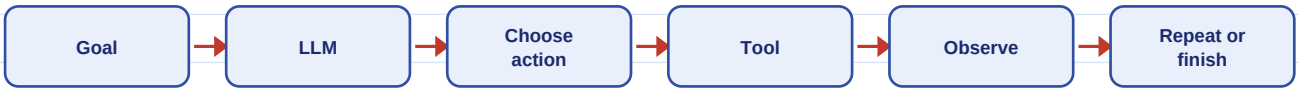
Move on when: My LLM correctly picks and uses the right tool.

My notes: _____

PHASE 8 - AI AGENTS

Time: 1 week

Started: _____ Completed: _____



- [ ] **1. What is an agent**
  An LLM that works toward a goal in a loop: choose action, use tool, observe result, decide next step.
- [ ] **2. ReAct pattern**
  Reason about what to do, Act using a tool, Observe the result, then repeat.
- [ ] **3. Planning**
  Breaking a goal into steps before or while acting.
- [ ] **4. Tool selection & observations**
  Choosing the right tool each step and using the returned result to decide what next.
- [ ] **5. Stopping conditions**
  Max steps, goal reached or confidence level so the agent doesn't loop forever.
- [ ] **6. Agent state & error handling**
  Track what has been done so far and recover from tool failures.

Build: AI Research Agent: search, read, search again, compare, summarize.

Move on when: I can explain and build a simple agent loop from scratch.

My notes: _____

PHASE 9 - LANGGRAPH *

Time: 1-1.5 weeks

** Invest heavily*

Started: _____ Completed: _____

- [ ] **1. State**
  A shared object (query, documents, messages, tool_results, final_answer) passed through the workflow.
- [ ] **2. Nodes**
  Functions that do one job each: retrieve, search, evaluate, generate.

- [ ] **3. Edges**
  Fixed connections deciding which node runs next.
- [ ] **4. Conditional edges & loops**
  Choose the next node based on state, e.g. if documents are poor then retrieve again.
- [ ] **5. Checkpoints & persistence**
  Save state at each step so workflows can resume or be inspected.
- [ ] **6. Streaming**
  Send partial outputs and step updates to the UI while the graph runs.
- [ ] **7. Human-in-the-loop & interrupts**
  Pause the graph, wait for human approval, then continue.

Build: Router -> RAG or Tools -> Evaluate -> good enough? -> answer or retrieve again.

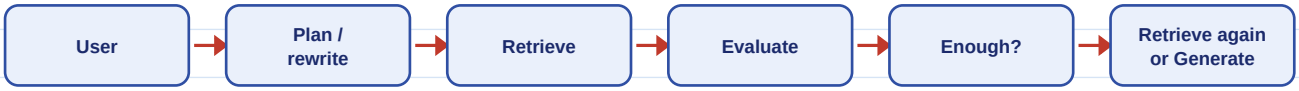
Move on when: I can design a multi-step agent on paper before coding it.

My notes: _____

PHASE 10 - AGENTIC RAG * — Time: 1 week

** Combine everything*

Started: _____ Completed: _____



- [ ] **1. Query planning**
  The agent decides whether retrieval is needed and how to split or rewrite the question.
- [ ] **2. Retrieval routing**
  Choose the right source: documents, database, or web.
- [ ] **3. Retrieval evaluation**
  Check if retrieved chunks actually answer the question.
- [ ] **4. Iterative retrieval**
  If information is missing, rewrite the query and retrieve again.
- [ ] **5. Tools + RAG combination**
  Use documents for knowledge and tools for live actions or data.
- [ ] **6. Web search + RAG / Database + RAG**
  Mix external search and SQL results with document retrieval for a complete answer.

Build: Project / Document AI Agent using your own project data.

Move on when: My system decides when and how to retrieve instead of always retrieving.

My notes: _____

PHASE 11 - MEMORY + HUMAN-IN-THE-LOOP — Time: 4-5 days

Started: _____ Completed: _____

- [ ] **1. Short-term memory**
  The current conversation history inside the context window.
- [ ] **2. Long-term memory**
  User facts and preferences saved in a database and loaded in later sessions.

- [ ] **3. Approval workflows**
  Sensitive actions (delete, update, send) pause and wait for user approval.
- [ ] **4. Interrupts & resumable execution**
  Stop at a checkpoint and continue exactly from there after approval.

Build: Agent that needs human approval for sensitive actions.

Move on when: I can explain history vs persistent memory and pause/resume an agent.

My notes: _____

PHASE 12 - EVALUATION + OBSERVABILITY

Time: 1 week

Started: _____ Completed: _____

- [ ] **1. RAG evaluation**
  Measure retrieval relevance, context precision, context recall, answer faithfulness and answer relevance.
- [ ] **2. Agent evaluation**
  Measure tool selection accuracy, task completion, trajectory quality and failure rate.
- [ ] **3. Tracing (LangSmith)**
  See every step: query, agent, tool, retriever, LLM, answer. Essential for debugging.
- [ ] **4. Datasets & experiments**
  Create test question sets and compare versions of your prompts or pipeline.
- [ ] **5. Latency, token usage & cost**
  Track speed and spending per request.

Build: Add a test dataset and tracing to your RAG/agent project.

Move on when: I can answer: how do I know my AI system actually works?

My notes: _____

PHASE 13 - AI SECURITY

Time: 4-5 days

Started: _____ Completed: _____

- [ ] **1. Prompt injection**
  Malicious instructions ('ignore all previous instructions') that try to override your prompt.
- [ ] **2. Indirect prompt injection**
  The attack hidden inside documents or web pages the agent reads.
- [ ] **3. Data leakage & PII**
  Prevent exposure of private documents, other users' data and personal information.
- [ ] **4. Tool security & authorization**
  LLM -> Tool -> Authorization -> Validation -> Execution. Never let the LLM run dangerous actions freely.
- [ ] **5. Input / output validation**
  Validate what goes into prompts and tools and what comes out of the model.
- [ ] **6. Secrets management & rate limiting**
  Protect API keys and limit abuse and cost.

Build: Add injection tests and permission checks to your agent.

Move on when: I can list the main attacks on my system and how I block them.

My notes: _____

PHASE 14 - PRODUCTION GENAI

Time: About 1 week

Started: _____ Completed: _____

- [ ] **1. Backend patterns**
  Background jobs/queues, streaming, retries, timeouts and rate limits in FastAPI.
- [ ] **2. Token optimization & caching**
  Reduce cost and latency by trimming context and caching repeated results.
- [ ] **3. Fallbacks & cost monitoring**
  Switch to a backup model on failure and watch spending.
- [ ] **4. Deployment**
  Frontend on Vercel, backend on Render/Railway, PostgreSQL database. Skip Kubernetes.

Build: Deploy your AI application publicly.

Move on when: Someone else can use my deployed AI app.

My notes: _____

PHASE 15 - MULTI-AGENT SYSTEMS

Time: After everything above

Started: _____ Completed: _____

- [ ] **1. Supervisor architecture**
  A manager agent assigns work to specialist agents and combines their results.
- [ ] **2. Specialist agents & delegation**
  Researcher, coder, reviewer: each with its own tools and instructions.
- [ ] **3. Shared state & agent communication**
  How agents pass information and results to each other.
- [ ] **4. Parallel vs sequential workflows**
  Run independent agents at the same time or chain them in order.
- [ ] **5. When NOT to use multi-agent**
  A single agent + tools + LangGraph is often simpler and better.

Build: Convert part of your project into a supervisor + specialist design only if it adds value.

Move on when: I can justify when multi-agent is worth it.

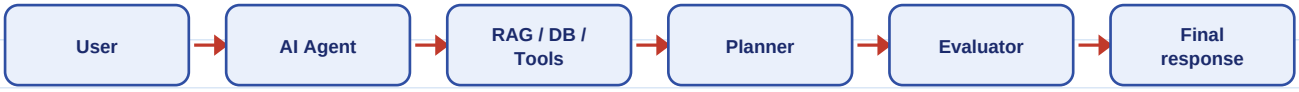
My notes: _____

PHASE 16 - MAJOR PROJECT *

Time: 3-4 weeks

Started: _____ Completed: _____

Build ONE serious project instead of ten small ones: an **AI-powered Project Intelligence / Management Agent** that combines everything above.



- [ ] **RAG** - Project documents, guidelines, reviews and project history.
- [ ] **Tools** - get_project(), get_deadline(), get_review(), create_task(), update_project().
- [ ] **Agent decisions** - Decides whether to search documents, query the database or call a tool.

- [ ] **Citations** - Every answer shows its source, e.g. [Project Guidelines, p.4].
- [ ] **Human approval** - Required for destructive or sensitive operations.
- [ ] **Evaluation & observability** - Measure retrieval quality, answer quality, task completion and tool accuracy; trace every agent decision.

My notes:

Final Progress Tracker

Phase	Topic	Done	Date	Confidence (1-5)
0	Python for GenAI Backend			
1	LLM Fundamentals			
2	Prompt Engineering + Structured Output			
3	Embeddings + Vector Search			
4	RAG			
5	Advanced RAG			
6	LangChain			
7	Tool Calling			
8	AI Agents			
9	LangGraph			
10	Agentic RAG			
11	Memory + Human-in-the-Loop			
12	Evaluation + Observability			
13	AI Security			
14	Production GenAI			
15	Multi-Agent Systems			
16	Major Project			

Golden rule: I can build this without following the tutorial.